

Bias-Variance Decomposition

ECMM422

Fabrizio Costa

Winter 2026



No Free Lunch Theorem

What is the No Free Lunch Theorem?

- ▶ The theorem states: **No algorithm performs universally better than others when averaged across all possible problems.**
- ▶ Originated from optimization and machine learning research by **Wolpert and Macready (1990s)**.
- ▶ **Intuition:** Every algorithm has strengths and weaknesses based on the problem.
- ▶ **Implication:** Tailored algorithms are necessary for specific problem types.

Toy Universe Example: Prediction and Performance

- ▶ **Setup:** A toy universe exists for exactly two days, each day containing one object: a square or a triangle.
- ▶ **Possible Histories:**
 - ① (square, triangle): Day 1 is a square, Day 2 is a triangle.
 - ② (square, square): Day 1 and Day 2 are both squares.
 - ③ (triangle, triangle): Day 1 and Day 2 are both triangles.
 - ④ (triangle, square): Day 1 is a triangle, Day 2 is a square.
- ▶ **Prediction Strategy:**
 - ▶ Predict a square on Day 2 if Day 1 contains a square.
 - ▶ Succeeds for history #2 but fails for history #1.
- ▶ **Key Insight:** Any prediction strategy will fail on exactly half of the histories (accuracy = 0.5) if all histories are equally likely.

Key Insight of the NFL Theorem

- ▶ Assumes a uniform distribution over all possible problems.
- ▶ **Symmetry of Performance:** Averaged over all problems, all algorithms perform equally.
- ▶ **Practical Insight:** Real-world problems are *not* uniformly distributed.
- ▶ Leads to the need for domain knowledge and problem-specific strategies.

Why Does It Matter?

- ▶ Highlights the **limitations of general-purpose algorithms.**
- ▶ Encourages the design of **specialized solutions.**
- ▶ Prevents overconfidence in "state-of-the-art" methods for every scenario.
- ▶ **Question:** How can this impact machine learning applications?

Formal Definition of the NFL Theorem

- ▶ For two algorithms, A and B , solving a set of problems $\mathcal{X}$:

$$\text{Performance of } A = \text{Performance of } B \text{ (averaged over all } \mathcal{X} \text{)}. \quad (1)$$

- ▶ **Assumption:** All problems are equally likely.
- ▶ **Implication:** Performance depends entirely on the problem distribution.

Key Assumption: Uniform Distribution

- ▶ Uniform distribution implies all problems have **equal probability**.
- ▶ Real-world problems are structured and non-uniform.
- ▶ **Challenge:** Algorithms must exploit patterns in real-world data.
- ▶ **Takeaway:** Domain knowledge breaks the NFL symmetry.

Real-World Application Examples

- ▶ Decision Trees: Effective for tabular data.
- ▶ Neural Networks: Perform well with large, complex data.
- ▶ **Challenge:** Matching the algorithm to the task and dataset.
- ▶ Example: Misusing deep learning on small datasets.

NFL Theorem: Problem Setup

- ▶ Let $\mathcal{X}$ be the set of all possible inputs and $\mathcal{Y}$ the set of all possible outputs.
- ▶ A **problem** is a mapping $f : \mathcal{X} \rightarrow \mathcal{Y}$.
- ▶ An **algorithm** A maps a training dataset $D = \{(x_i, y_i)\}$ to a hypothesis h_A . This hypothesis is used to predict y for unseen x .
- ▶ **Performance Metric:** The error $E_A(f)$ for algorithm A on problem f is defined as:

$$E_A(f) = \sum_{x \in \mathcal{X}} \mathbb{I}(h_A(x) \neq f(x)), \quad (2)$$

where $\mathbb{I}(\cdot)$ is the indicator function.

NFL Theorem: Core Argument

- ▶ Consider the average error of an algorithm A over all possible problems f .
- ▶ For any pair of algorithms A and B :

$$\frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} E_A(f) = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} E_B(f), \quad (3)$$

where $\mathcal{F}$ is the set of all possible problems.

- ▶ This equality holds because:
 - ▶ Each f is equally likely.
 - ▶ There is no preferential structure in $\mathcal{F}$ for any algorithm.
- ▶ **Implication:** No algorithm can outperform another on average over all f . One such algorithm is: predict at random (!)

Matrix of Binary Functions: Inputs and Outputs

► Binary Function Setup:

- Input space: $\mathcal{X} = \{x_1, x_2\}$
- Output space: $\mathcal{Y} = \{0, 1\}$

► All Possible Functions:

Input	x_1	x_2	f_1	f_2	f_3	$\dots$	f_n
Output	0	0	1	0	1	$\dots$	1
	0	1	0	1	1	$\dots$	0
	1	0	1	1	0	$\dots$	1
	1	1	0	0	1	$\dots$	0

► NFL Implication:

- Each f_i represents a different problem.
- Any algorithm optimized for one f_i fails equally on others.
- Averaged over all f_i , no algorithm has an advantage.

Equivalent Formulation of the NFL Theorem (1/2)

- ▶ **Theorem:** Let V be a finite set and S a finite set of real numbers. Assume $f : V \rightarrow S$ is chosen randomly with uniform distribution on the set S^V . For optimizing f over V :
 - ▶ No algorithm performs better than **blind search**.
- ▶ **Blind Search:**
 - ▶ At each step, an element $v \in V$ is chosen randomly with uniform probability from V .
 - ▶ Ensures no prior knowledge or pattern exploitation.
- ▶ **Key Insight:** All algorithms perform equally well on average if f is uniformly random.

Equivalent Formulation of the NFL Theorem (2/2)

► Implication:

- The probability of observing any sequence of m values during optimization is independent of the algorithm.
- Performance depends only on the sequence of observed values, not on the time taken or the strategy used.

► Key Consequence:

- When f is drawn uniformly, all algorithms have **identical mean performance**.
- No algorithm has an intrinsic advantage.

- **Conclusion:** In a uniformly random landscape, algorithm performance is indistinguishable from random sampling.

- ▶ **Generalization vs. Specialization:**

- ▶ Generalization: Performing well on unseen data.
- ▶ Specialization: Optimizing for specific problem classes.

- ▶ **Domain knowledge** helps break the NFL assumptions.

- ▶ Example: Neural networks excel at unstructured data like images.

Algorithm Selection and Trade-offs

- ▶ Different algorithms excel under different conditions.
- ▶ **Bias-Variance Trade-off:** Balancing overfitting and underfitting.
- ▶ **Takeaway:** No single model is suitable for all tasks.
- ▶ **Question:** How can we evaluate algorithm performance effectively?

Bias Variance Theory

- ▶ We want to study the behavior of predictive systems
- ▶ We want to find relations between the error and other meaningful concepts
- ▶ We will consider the **bias**, i.e. the error due to the difference between the *true* function and the *model* function that we can represent/compute
- ▶ ..and the **variance**, i.e. the estimation error due to having a finite sample to train out models with

Expected Value

- ▶ The **expected value** or *mean* or *average* of a random variable x is:

$$E[x] = \sum_{x \in \mathcal{X}} xp(x)$$

- ▶ The expected value is denoted with the symbol μ
- ▶ *If one thinks of probabilities as weights, then the expected value is the center of mass*
- ▶ Given a function $f(x)$ we define:

$$E[f(x)] = \sum_{x \in \mathcal{X}} f(x)p(x)$$

Expected Value is a Linear Operator

- ▶ The process of computing the expected value is linear
- ▶ Given two arbitrary constants α_1 and α_2 , it holds that:

$$E[\alpha_1 f_1(x) + \alpha_2 f_2(x)] = \alpha_1 E[f_1(x)] + \alpha_2 E[f_2(x)]$$

- ▶ We can think of E as a **linear operator**

- ▶ The **second moment** is:

$$E[x^2] = \sum_{x \in \mathcal{X}} x^2 p(x)$$

- ▶ The **variance** is:

$$E[(x - E[x])^2] = E[(x - \mu)^2] = \sum_{x \in \mathcal{X}} (x - \mu)^2 p(x)$$

- ▶ The variance is indicated with σ^2 where σ is the **standard deviation**
- ▶ The variance is also indicated as $Var(x)$

Notes on the standard deviation

- ▶ The standard deviation measures how much the values of x tend to differ from their average μ
- ▶ For a normal distribution we have:
 - ▶ 68% of values are within 1σ
 - ▶ 95% of values are within 2σ
 - ▶ 99.7% of values are within 3σ
- ▶ In the general case a (loose) bound is given by the **Chebyshev's inequality**
- ▶ **Theorem:** for a wide class of probability distributions, no more than a certain fraction of values can be more than a certain distance from the mean.

$$p(|x - \mu| > n\sigma) \leq \frac{1}{n^2}$$

- ▶ It holds that:

$$\sigma^2 = E[(x - E[x])^2] = E[x^2] - (E[x])^2$$

- ▶ Proof: let's use the shorthand $\bar{x} = E[x]$

$$E[(x - \bar{x})^2] = E[x^2 - 2x\bar{x} + \bar{x}^2] \quad (4)$$

$$= E[x^2] - 2E[x]\bar{x} + \bar{x}^2 \quad (5)$$

$$= E[x^2] - 2\bar{x}^2 + \bar{x}^2 \quad (6)$$

$$= E[x^2] - \bar{x}^2 \quad (7)$$

- ▶ and consequently:

$$E[x^2] = E[(x - \bar{x})^2] + \bar{x}^2$$

- ▶ Between two random variables it holds that:

$$\sigma(x, y) = E[(x - E[x])(y - E[y])] = E[xy] - E[x]E[y]$$

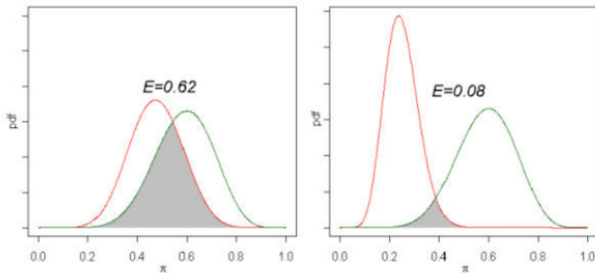
- ▶ $\sigma(x, y)$ is called the *covariance*
- ▶ Proof: again we use the shorthand $\bar{x} = E[x]$ and $\bar{y} = E[y]$

$$\begin{aligned} E[(x - E[x])(y - E[y])] &= E[xy - \bar{x}y - x\bar{y} + \bar{x}\bar{y}] \\ &= E[xy] - \bar{x}E[y] - \bar{y}E[x] + \bar{x}\bar{y} \\ &= E[xy] - \bar{x}\bar{y} - \bar{y}\bar{x} + \bar{x}\bar{y} \\ &= E[xy] - \bar{x}\bar{y} \end{aligned}$$

- ▶ note that if x and y are independent then $p(xy) = p(x)p(y)$ and hence $E[xy] = E[x]E[y]$ and finally $\sigma(x, y) = 0$

Bias Variance Theory

- ▶ **Aim:** decompose the error of a predictive system induced from a finite data sample into meaningful components for insight
- ▶ **Result:** error = irreducible (Bayes) error + bias^2 + variance



Bias Variance Theory: regression

- ▶ Assume the true function can be written as:

$$y = f(x) + \epsilon$$

- ▶ where $\epsilon = N(0, \sigma^2)$, i.e. ϵ is a random variable normally distributed with zero mean and standard deviation σ
- ▶ We sample examples (x, y) from an unknown probability distribution to compose datasets of finite size M
- ▶ We consider one such set of training examples $S = \{(x_i, y_i)\}_{i=1}^M$
- ▶ On this data set we **fit** $h(\cdot)$, i.e. our model function or **hypothesis**
- ▶ For example:
 - ▶ we can choose the class of linear functions $h(x) = x \cdot \beta$
 - ▶ ..and as fitting criterion, we can minimize the squared error $\sum_i (y_i - h(x_i))^2$

Bias Variance Theory: regression

- ▶ We are given a new data point x_0 extracted from the same population from which we extracted the sets of training examples
- ▶ We observe the corresponding value

$$y_0 = f(x_0) + \epsilon$$

- ▶ We want to understand and decompose the **expected prediction error**

$$E[(y_0 - h(x_0))^2]$$

- ▶ **Note:** the expectation is taken over all datasets of size M : for each dataset we fit $h()$ from the specified family of functions using the stated fitting criterion and procedure

Bias Variance Theory: decomposition

So, given that: $y_0 = f(x_0) + \epsilon$

$$E[(y_0 - h(x_0))^2] = E[(y_0 - f(x_0) + f(x_0) - h(x_0))^2] \quad (8)$$

$$\begin{aligned} &= E[(y_0 - f(x_0))^2] + E[(f(x_0) - h(x_0))^2] \\ &\quad + 2E[(y_0 - f(x_0))(f(x_0) - h(x_0))] \end{aligned} \quad (9)$$

$$\begin{aligned} &= E[(f(x_0) + \epsilon - f(x_0))^2] + E[(f(x_0) - h(x_0))^2] \\ &\quad + 2(E[y_0 f(x_0)] - E[y_0 h(x_0)] \\ &\quad - E[f(x_0)^2] + E[f(x_0)h(x_0)]) \end{aligned} \quad (10)$$

$$= E[\epsilon^2] + E[(f(x_0) - h(x_0))^2] + 0 \quad (11)$$

Bias Variance Theory: decomposition

We have to show that:

$$2(E[y_0 f(x_0)] - E[y_0 h(x_0)] - E[f(x_0)^2] + E[f(x_0)h(x_0)]) = 0$$

in fact:

$$E[y_0 f(x_0)] = f(x_0)E[y_0] = f(x_0)E[f(x_0) + \epsilon] = f(x_0)^2 + 0$$

which cancels out with $-E[f(x_0)^2] = -f(x_0)^2$

Note: $f(x_0)$ is constant and $E[\epsilon] = 0$

finally:

$$-E[y_0 h(x_0)] = -E[(f(x_0) + \epsilon)h(x_0)] = -E[f(x_0)h(x_0)] - E[\epsilon h(x_0)]$$

where $-E[f(x_0)h(x_0)]$ cancels out with $E[f(x_0)h(x_0)]$

and $-E[\epsilon h(x_0)] = -E[\epsilon]E[h(x_0)] = 0 \cdot E[h(x_0)] = 0$

Note: if two random variables are independent then

$E[ab] = E[a]E[b] + Cov[ab] = E[a]E[b]$ since $Cov[ab] = 0$, and here the noise is independent from our hypothesis

Bias Variance Theory: decomposition

From eq. (8) we had the term:

$$\begin{aligned} E[(f(x_0) - h(x_0))^2] &= E[(f(x_0) - \bar{h}(x_0) + \bar{h}(x_0) - h(x_0))^2] \\ &= E[(f(x_0) - \bar{h}(x_0))^2] + E[(\bar{h}(x_0) - h(x_0))^2] \\ &\quad + 2E[(f(x_0) - \bar{h}(x_0))(\bar{h}(x_0) - h(x_0))] \\ &= E[(f(x_0) - \bar{h}(x_0))^2] + E[(\bar{h}(x_0) - h(x_0))^2] + 0 \\ &= (f(x_0) - \bar{h}(x_0))^2 + E[(\bar{h}(x_0) - h(x_0))^2] \end{aligned}$$

where $\bar{h}(x_0) = E[h(x_0)]$, i.e. this is the average prediction for x_0 over all training sets

Bias Variance Theory: decomposition

We have to show that:

$$E[(f(x_0) - \bar{h}(x_0))(\bar{h}(x_0) - h(x_0))] = 0$$

$$E[(f(x_0)\bar{h}(x_0))] - E[f(x_0)h(x_0)] - E[\bar{h}(x_0)^2] + E[\bar{h}(x_0)h(x_0)] = 0$$

in fact:

$E[(f(x_0)\bar{h}(x_0))] = f(x_0)\bar{h}(x_0)$, since $f(x_0)$ is constant
which cancels out with $-E[f(x_0)h(x_0)] = -f(x_0)E[h(x_0)] = -f(x_0)\bar{h}(x_0)$
finally, $E[\bar{h}(x_0)^2] = \bar{h}(x_0)^2$, since $\bar{h}(x_0)$ is constant
which cancels out with $E[\bar{h}(x_0)h(x_0)] = \bar{h}(x_0)E[h(x_0)] = \bar{h}(x_0)^2$

Bias Variance Theory: decomposition

We can finally put everything together:

$$\begin{aligned} E[(y_0 - h(x_0))^2] &= E[\epsilon^2] \\ &\quad + (f(x_0) - \bar{h}(x_0))^2 \\ &\quad + E[(h(x_0) - \bar{h}(x_0))^2] \end{aligned} \tag{12}$$

$$= \sigma^2 + \text{Bias}(h(x_0))^2 + \text{Var}(h(x_0)) \tag{13}$$

$$\tag{14}$$

Expected prediction error = Noise + Bias² + Variance

Bias Variance Theory: decomposition

- ▶ **Noise:** $E[(y_0 - f(x_0))^2] = \sigma^2$
it describes how much y_0 can differ from the true $f(x_0)$ due to intrinsic uncertainties
- ▶ **Bias:** $(\bar{h}(x_0) - f(x_0))^2$
it describes the difference between the expected predicted value and the true (but unknown) value
- ▶ **Variance:** $E[(h(x_0) - \bar{h}(x_0))^2]$
it describes the variability of the prediction $h(x_0)$ when different training sets are used to fit the model h

Measuring Bias and Variance

► **Problem:**

To measure the *expected prediction error* we need to induce predictors from many training sets

► ...unfortunately we generally have only **one** training set

► **Solution:** simulate multiple training sets by **bootstrap replicates**

① Given a single set of training examples $S = \{(x_i, y_i)\}_{i=1}^M$

② Build several replicate data sets, every time using the following procedure:

$S' = \{x | x \text{ is drawn at random with replacement from } S\}$

③ continue drawing until the replicate has the same size $|S'| = |S|$

Measuring Bias and Variance

- 1 Make B bootstrap replicates of S : $S_1, \dots, S_B$
- 2 Use S_b as training set and induce hypothesis h_b
- 3 Make **out of bag** set $T_b = S \setminus S_b$
i.e. T_b is made of all instances that do **not** appear in S_b
- 4 Compute $h_b(x)$ for each x in T_b (indicate with K their number)
Note that the same instance x appears in different T_b

Measuring Bias and Variance

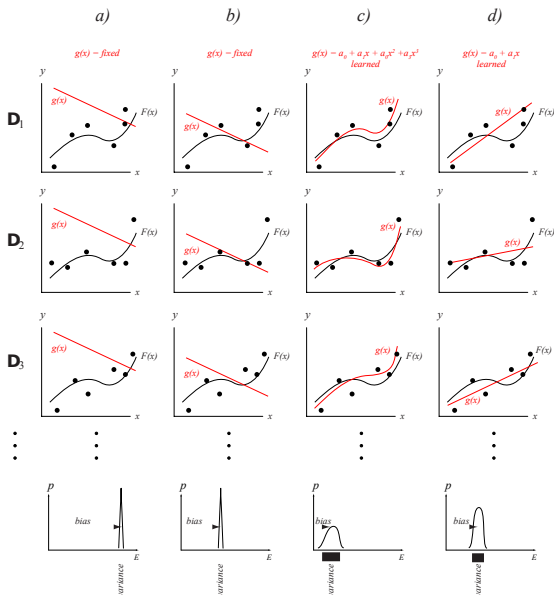
- ➊ Compute expected prediction $\bar{h}(x) = \frac{1}{K} \sum_{b=1}^K h_b(x)$
 - ➋ Estimate bias² as $(\bar{h}(x) - y)^2$
 - ➌ Estimate variance as $\frac{1}{K-1} \sum_{b=1}^K (\bar{h}(x) - h_b(x))^2$
 - ➍ Assume noise is 0
- ▶ Note that we are ignoring the noise
 - ▶ If we have multiple pairs (x_i, y_i) for the same value x_i then we can also estimate the noise
 - ▶ Alternatively we can estimate it by considering the y values associated to near/similar x

Bias Variance Dilemma

In the experimental practice we observe an important phenomenon called the **bias variance dilemma** or *bias variance trade-off*

- ➊ Given two classes of hypothesis (e.g. linear models and k -NNs) to fit to some training data set
... we observe that the more flexible hypothesis class generally exhibits a low bias term, but also, inevitably, a higher variance term
- ➋ If we have a single parametric family of hypothesis with a parameter that controls the complexity (e.g. k -NN for different values of k), then when the flexibility of the hypothesis is increased (e.g. reducing k) we generally observe also an increase of variance

Bias Variance Example 1D



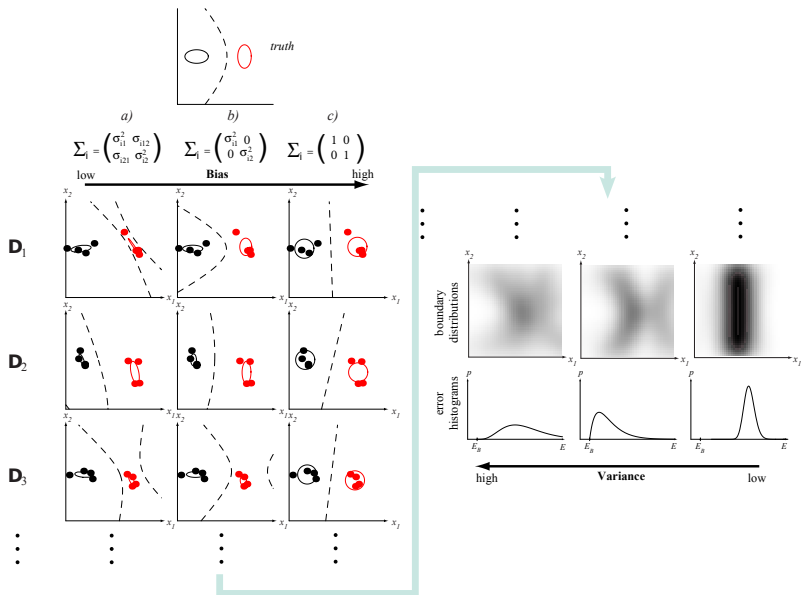
Bias Variance Example 1D

- ▶ Each column is a different model: we have two constant functions, a cubic polynomial and a linear model
- ▶ Each row is a training set made of 6 points sampled from the same true function (a cubic polynomial) with noise
- ▶ Last row shows the error histogram: error in the horizontal axis and the probability of error in the vertical axis

Bias Variance Example 1D

- ▶ **Observation 1:** the constant models have a large bias but zero variance; the second model has a lower bias than the first
- ▶ **Observation 2:** the cubic model has the lowest bias but the highest variance
- ▶ **Observation 3:** the linear model has a low variance, i.e. very similar predictors are obtained from different training sets
- ▶ If we had $n \rightarrow \infty$ then variance for all models would vanish; however only the bias for the cubic model would diminish (up to the intrinsic Bayes noise level)

Bias Variance Example 2D

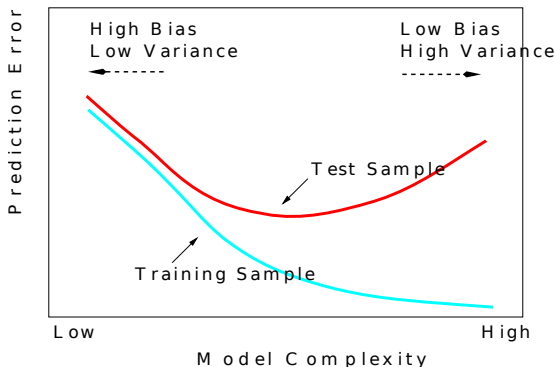


Bias Variance Example 2D

- ▶ Each column is a different model from the same family of functions: we have Gaussian with full, diagonal and unit covariance matrix
- ▶ Each row is a training set made of 8 points sampled from the same true function (two Gaussians)
- ▶ On the right the decision boundary distribution and the error histogram: error in the horizontal axis and the probability of error in the vertical axis
- ▶ **Observation:** the trade-off between bias and variance is once again consistent: low bias $\Leftrightarrow$ high variance

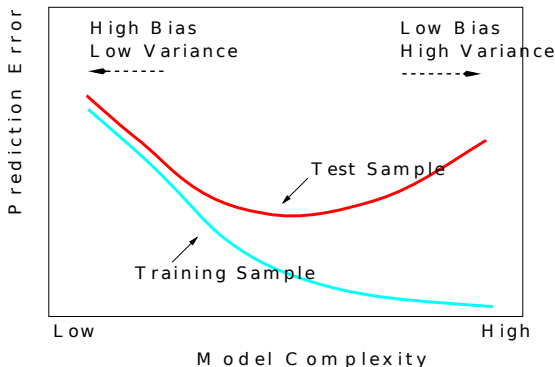
- ▶ The expected prediction error is the sum of a bias component and a variance component
- ▶ To obtain a low error it is generally **better** to prefer low variance to low bias
- ▶ For a given bias the variance can be **diminished** by increasing the size of the training set
- ▶ The bias can be reduced increasing the complexity of the model until a perfect match with the true underlying function is achieved
- ▶ The error cannot however be reduced below the Bayes Error
- ▶ The only way to have zero bias and zero variance is to use the correct true function (by guessing it without any learning)

Expected prediction error estimation



- ▶ The expected prediction error (EPE) can be estimated from the error on an independent test set
- ▶ The error on the training set instead is a **optimistic estimate** of EPE as it does not take into account the model complexity (i.e. this error always decreases with higher complexity)

Expected prediction error estimation



- ▶ The training error does not allow to estimate the variance component of the error
- ▶ A large variance implies a large EPE
- ▶ How can we obtain a good estimate of EPE?

Cross-Validation

- ▶ If we had enough data we could set aside a large set to estimate the error
- ▶ In practice we have a finite sample of data that we have to split between training and validation data
- ▶ If we use a large portion for validation we do not have enough data for training
- ▶ If we use a large portion for training we do not estimate the error accurately
- ▶ A compromise is to use the **K-fold cross-validation** technique

- 1 Randomly permute the data
- 2 Split the data into K roughly equal sized parts with the same distribution of targets as in the whole set
- 3 Let h^{-i} be the hypothesis fitted to the data set with the i -th part removed
- 4 The cross-validation estimate of the prediction error is:

$$CV(h) = \frac{1}{M} \sum_{i=1}^M L(y_i, h^{-i}(x_i))$$

for some loss function $L()$

Cross-Validation

The cross-validation procedure can be used to **select** the model with lowest expected prediction error

- 1 Divide the data set as before
- 2 Let $h(x, \alpha)$ be a hypothesis for point x under parameter α (i.e. α can be the number of neighbors in the k -NN model)
- 3 Let h^{-i} be the hypothesis fitted to the data set with the i -th part removed
- 4 The cross-validation estimate of the prediction error of h with parameter α is:

$$CV(h, \alpha) = \frac{1}{M} \sum_{i=1}^M L(y_i, h^{-i}(x_i, \alpha))$$

for some loss function $L()$

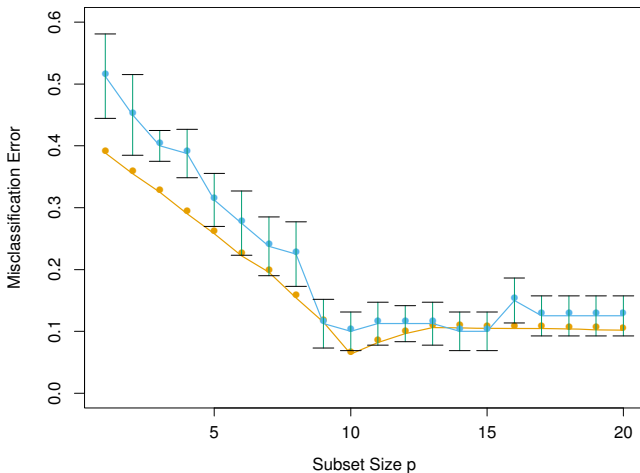
- 5 Choose the model parameter α^* that minimizes CV

Choice of Cross-Validation parameter K

The estimation of the error by the cross-validation procedure is itself subject to the bias-variance trade-off

- ▶ If $K = M^1$ is an unbiased estimator of the true error but has a large variance (because all the training sets are very similar to each other)
- ▶ If $K = 5$ the variance is low but the bias can be high (a predictor that overfits a small subset of the data will consistently exhibit an optimistically low error)
- ▶ $K = 10$ is recommended as a good compromise

¹This case is also known as **Jackknife**



Example of the selection of model parameter p with 10-CV.

One can use the *one-standard error rule*: choose the model with the smallest complexity whose CV error is no more than 1 std above the best.

The Wrong and the Right Way of Doing Cross-Validation

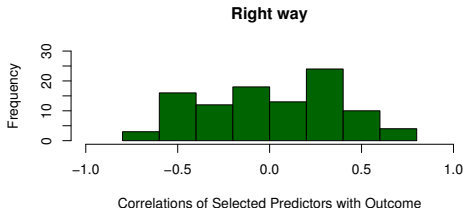
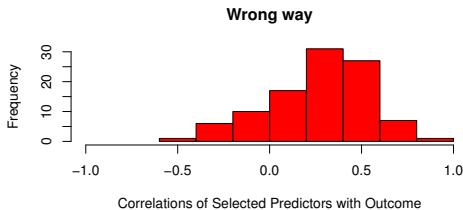
- ▶ Consider a classification problem with many features (e.g. genomic application)
- ▶ This could be a strategy used to build a predictor
 - ▶ Filter the features: find a subset of features that correlates strongly with the class label
 - ▶ Use only those features to build a predictor
 - ▶ Perform cross-validation to estimate the tuning parameters and the prediction error
- ▶ Is this procedure correct?

The Wrong and the Right Way of Doing Cross-Validation

- ▶ To see why the previous approach is wrong consider this experiment:
 - ▶ Take 50 samples of a binary classification problem with $p = 5000$ features
 - ▶ The values for each feature are sampled from Gaussian distributions
 - ▶ The class label is **independent** of any feature
 - ▶ In this conditions any predictor has a true test error of 50%
 - ▶ Choose 100 features with the highest correlation to the class label
 - ▶ Use a 1-NN predictor based on these features

The Wrong and the Right Way of Doing Cross-Validation

- ▶ **Q:** If this procedure is repeated 50 times, what will be the average CV error estimate?
- ▶ **A:** Surprisingly it can be 3% instead of 50%



The Wrong and the Right Way of Doing Cross-Validation

- ▶ The features have been selected looking at all the data
- ▶ The information on the class of the test point was available during the training procedure
- ▶ The construction of the predictor happens in reality in 2 steps: 1) choice of the features and 2) fitting of the model's parameter
- ▶ The CV estimate has to take into consideration the whole process

The Wrong and the Right Way of Doing Cross-Validation

- ▶ What is the correct way of doing cross-validation?
- ▶ Divide initially the dataset into the K parts
- ▶ In each fold independently:
 - ▶ Select the features that correlate best with the target
 - ▶ Fit a predictor using those features
 - ▶ Use the predictor to classify the instances in the validation fold
- ▶ Accumulate the error estimate over the K folds and report the average CV error estimate

Note: *One can correctly use a criterion that does not access the label information to filter the features (e.g. those with highest variance) before doing the K parts split*